# Deliverable-2

October 27, 2026

## 0.1 Deliverable 2

### 0.1.1 Jon Gundesø og Jonas Kristoffersen

### 0.1.2 Task 1

[93]:
```
import re
from pathlib import Path


def file_reader(path_name: str) -> tuple[str, str]:
    """It reads a file and return a tuple with encoding and text"""
    # we have to read as bytes so that python doenst decode the file
    file = path_name.read_bytes()
    # we check the encoding
    try:
        encoding = "utf-8"
        text = file.decode(encoding)
    except UnicodeDecodeError:
        encoding = "iso-8859-1"


    # we read the context of the file using the encodeing we found
    text = path_name.read_text(encoding=encoding)
    return text, encoding

def log_parser(text : str) -> list[dict["timestamp" : str, "register_id" : str,␣
 ↪"item" : str, "price" : float, "currency" : str, "status" : str]]:
    parsed_logs = []
    """Parsers logs into into the 6 key fields and remove cancelled␣
 ↪transection"""
    # first we need to splitt the transaction record into individual␣
 ↪transactions
    not_wanted = "--- PRINTER PAPER LOW ---\n|=== NETWORK SYNC OK ===\n"
    clean_text = re.sub(not_wanted, "", text)
    transactions = re.split("\n", clean_text)
    # then we get the data we want for each line
    for i in transactions:
```

```
        # got some help from ai to make a pattern that groups all the values we want
        monster = r"\[(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\] \[(?P<register_id>POS_\d+)\] item=(?P<item>\w+) price=(?P<price>[\d\.]+) (?P<currency>\w+) \((?P<status>\w+)\)"
        # we search for our pattern in the string
        match = re.search(monster, i)
        # we check if it matches
        if match:
            # we add the group dictionary to the list
            parsed_logs.append(match.groupdict())

    filtered_list = [d for d in parsed_logs if d.get("status") != "CANCELLED"]

    return filtered_list
```

### 0.1.3 Task 2

[94]:
```
import json


def find_log_files(path: str | Path) -> list[Path]:
    """Find all .log files in a directory and its subdirectories."""
    path = Path(path)
    log_files = []

    for item in path.iterdir():
        if item.is_dir():
            log_files.extend(find_log_files(item))
        elif item.is_file() and item.suffix == ".log":
            log_files.append(item)

    return log_files


def find_activeID(data: dict | list) -> set[str]:
    """Find all register IDs where active is True."""

    active_registers = set()

    if isinstance(data, dict):
        if data.get("active") is True and "id" in data:
            active_registers.add(data["id"])

        for value in data.values():
            active_registers.update(find_activeID(value))
```

```
    elif isinstance(data, list):
        for item in data:
            active_registers.update(find_activeID(item))

    return active_registers
```

### 0.1.4 Task 3

[95]:
```
import numpy as np
import pandas as pd


def remove_outliers(df: pd.DataFrame) -> np.array:
    """The function remove value errors and values who is bigger than 3x SD"""
    # make a list for valid values to make a median value and a list that is a␣
 ↪copy of the log

    x = []
    y = []

    for i in df:
        values = float(i)
        if values < 900:
            # appends the values to both lists if the value is over 900
            x.append(values)
            y.append(values)

        else:
            # appends the value to the list with all values
            y.append(values)

    # calculate mean and standard deviation
    mean = np.mean(x)
    sigma = np.std(x)

    # make the list with all values into an array
    arr = np.array(y)
    # swap the error values for the mean value
    arr[arr == 999] = mean
    # swap the values that are more than 3 standard deviations away
    arr[arr - mean > 3 * sigma] = mean
    return arr



def compute_refund_ratio(succes_count, refund_count) -> np.ndarray:
    """Compute refund ration. Caount sales and refund inside the code,
```

```
    because we miss a function in the deliverable task."""
    # known values
    e = 10e-6

    # calculate the refund ratio
    Refund_ratio = (refund_count / succes_count + e)
    return Refund_ratio
```

### 0.1.5 Task 4

[96]:
```
import pandas as pd


def transactions_to_dataframe(transactions):
    """Convert transactions to a chronologically sorted DataFrame."""
    df = pd.DataFrame(transactions)
    if df.empty:
        return df

    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df = df.set_index("timestamp").sort_index()
    return df


def hourly_revenue_by_item(df):
    """Calculate total hourly revenue for each item."""

    hourly_revenue = (
        df.groupby("item")["price"]
        .resample("1h")
        .sum()
    )

    return hourly_revenue


def make_pivot(df: pd.DataFrame) -> pd.DataFrame:
    """Create a pivot table showing total revenue by item and status."""

    pivot = pd.pivot_table(
        df,
        index="item",
        columns="status",
        values="price",
        aggfunc="sum",
        fill_value=0
    )
```

```
    return pivot


def refund_ratio_by_item(df: pd.DataFrame) -> pd.DataFrame:
    """Calculate the refund ratio for each item."""

    items = sorted(df["item"].unique())

    success_count = (
        df[df["status"] == "SUCCESS"]["item"]
        .value_counts()
        .reindex(items, fill_value=0)
        .to_numpy(dtype=float)
    )

    refund_count = (
        df[df["status"] == "REFUND"]["item"]
        .value_counts()
        .reindex(items, fill_value=0)
        .to_numpy(dtype=float)
    )

    refund_ratios = compute_refund_ratio(success_count, refund_count)

    return pd.DataFrame({
        "item": items,
        "refund_ratio": refund_ratios
    })
```

### 0.1.6 Task 5

[97]:
```
# 1. Find log files
log_files = find_log_files("d2_data")

print("=== Log files found ===")
for log_file in log_files:
    print(log_file)

# 1.2. Find active id
with open("d2_data\\store_config.json", "r", encoding="utf-8") as file:
    config = json.load(file)

active_id = find_activeID(config)

print("\n=== Active registers ===")
for register_id in active_id:
```

```
    print(register_id)

# 2. Read log files and parse transactions
transactions = []

for log_file in log_files:
    text, encoding = file_reader(log_file)
    parsed_logs = log_parser(text)
    transactions.extend(parsed_logs)

print(f"\nNumber of transactions found: {len(transactions)}")

# 3.
df = transactions_to_dataframe(transactions)
# 4.
df["price"] = remove_outliers(df["price"])

# 5.
df.loc[df["currency"] == "NOK", "price"] *= 10.0


# 7. Create summary files
Transaction = df
hourly = hourly_revenue_by_item(df)
pivot = make_pivot(df)
refund_ratios = refund_ratio_by_item(df)

Transaction.to_csv("cleaned_transactions.csv")
hourly.to_json("hourly_revenue.json", indent=4)
pivot.to_csv("status_pivot_table.csv")
refund_ratios.to_csv("item_refund_ratios.csv")

# 6. summary
print(df)
print(hourly)
print(pivot)
print(refund_ratios)
```

```
=== Log files found ===
d2_data\2026\library_cafe\POS_01\register_sales.log
d2_data\2026\library_cafe\POS_02\register_sales.log
d2_data\2026\library_cafe\POS_03\register_sales.log
d2_data\2026\science_building\POS_01\register_sales.log
d2_data\2026\science_building\POS_02\register_sales.log
d2_data\2026\science_building\POS_03\register_sales.log
d2_data\2026\student_union\POS_01\register_sales.log
d2_data\2026\student_union\POS_02\register_sales.log
d2_data\2026\student_union\POS_03\register_sales.log
```

```
=== Active registers ===
POS_01
POS_02
POS_03

Number of transactions found: 587
                    register_id        item  price currency   status
timestamp
2026-10-15 07:00:43      POS_02       Latte   4.48      EUR  SUCCESS
2026-10-15 07:00:44      POS_01      Muffin   3.88      EUR  SUCCESS
2026-10-15 07:00:49      POS_03   Croissant   3.77      USD  SUCCESS
2026-10-15 07:00:57      POS_03    Sandwich   7.98      EUR   REFUND
2026-10-15 07:01:00      POS_01    Sandwich   7.51      EUR  SUCCESS
…                         …          …    …      …       …
2026-10-15 09:10:55      POS_02   Croissant  36.90      NOK  SUCCESS
2026-10-15 09:12:02      POS_01       Latte   4.39      EUR   REFUND
2026-10-15 09:14:19      POS_01      Muffin   3.54      EUR  SUCCESS
2026-10-15 09:17:17      POS_01  Cappuccino   3.77      USD  SUCCESS
2026-10-15 09:20:52      POS_01   Croissant   4.06      EUR  SUCCESS

[587 rows x 5 columns]
item        timestamp
Cappuccino  2026-10-15 07:00:00     981.758970
            2026-10-15 08:00:00     626.407644
            2026-10-15 09:00:00      16.690000
Croissant   2026-10-15 07:00:00     717.188970
            2026-10-15 08:00:00     468.987941
            2026-10-15 09:00:00     171.540000
Espresso    2026-10-15 07:00:00     575.550000
            2026-10-15 08:00:00     760.120000
            2026-10-15 09:00:00      59.830000
Latte       2026-10-15 07:00:00     730.718377
            2026-10-15 08:00:00     855.468970
            2026-10-15 09:00:00     113.370000
Muffin      2026-10-15 07:00:00     650.718970
            2026-10-15 08:00:00     591.138970
            2026-10-15 09:00:00      54.700000
Sandwich    2026-10-15 07:00:00     993.370000
            2026-10-15 08:00:00    1437.198970
            2026-10-15 09:00:00      37.060000
Name: price, dtype: float64
status         REFUND      SUCCESS
item
Cappuccino  315.49897  1309.357644
Croissant   257.70897  1100.007941
Espresso    255.39000  1140.110000
Latte       134.58000  1564.977347
```

```
Muffin      236.16000  1060.397941
Sandwich    568.90897  1898.720000
         item  refund_ratio
0  Cappuccino      0.237510
1   Croissant      0.207327
2    Espresso      0.202391
3       Latte      0.232568
4      Muffin      0.197378
5    Sandwich      0.300010
```

### 0.2 Gen AI Summary

We have used Ai to help with the code libraries we dont know that well,
like pandas dataframe and numpy array.
we have used Ai to help with debugging the code when we have met bugs.
we have used it to understand what the assignment asks of us to use as inputs and outputs, it was quite difficult to understand what the assignment wanted in each task.
we used it to help make the re module patterns for the group dicts, because it was new and difficult to do all of them in one pattern.